



# **Survey on Evolution of Java Web Technologies and Best Practices: from Servlets to Microservices**

**Avinash Attipalli <sup>a\*</sup>, Raghuvaran Kendyala <sup>b</sup>,  
Jagan Kurma <sup>c</sup>, Jaya Vardhani Mamidala <sup>d</sup>,  
Varun Bitkuri <sup>e</sup> and Sunil Jacob Enokkaren <sup>f</sup>**

<sup>a</sup> Department of Computer Science, University of Bridgeport, United States.

<sup>b</sup> Department of Computer Science, University of Illinois at Springfield, United States.

<sup>c</sup> Computer Information Systems, Christian Brothers University, United States.

<sup>d</sup> Department of Computer Science, University of Central Missouri, United States.

<sup>e</sup> Stratford University, United States.

<sup>f</sup> ADP, Solution Architect, United States.

## **Authors' contributions**

*This work was carried out in collaboration among all authors. All authors read and approved the final manuscript.*

## **Article Information**

DOI: <https://doi.org/10.9734/ajrcos/2025/v18i11786>

## **Open Peer Review History:**

This journal follows the Advanced Open Peer Review policy. Identity of the Reviewers, Editor(s) and additional Reviewers, peer review comments, different versions of the manuscript, comments of the editors, etc are available here: <https://pr.sdiarticle5.com/review-history/146611>

**Review Article**

**Received: 01/09/2025**  
**Published: 20/11/2025**

## **ABSTRACT**

The evolution of Java web technologies has had a significant impact on the development of modern web and enterprise application development. This paper provides a thorough review of the history of Java since its establishment as a server-side set of technologies up to the present-day architectures based on microservices. It starts with the analysis of the initial innovations like Servlets, Java Server Pages (JSP), and Java Database Connectivity (JDBC) that defined the

\*Corresponding author: E-mail: [attipalli.avinash@gmail.com](mailto:attipalli.avinash@gmail.com)

dynamic content generation, database integration, and simple web actions. It is then followed by the discussion of the emergence of enterprise frameworks, such as Struts, Spring, and Hibernate, which brought the concepts of modularity, scale, and better maintainability of applications with the separation of concerns. Further, the paper discusses the improvement of enterprise solutions by using Enterprise JavaBeans (EJB), the Java Persistence API (JPA), and by use of RESTful web services via JAX-RS, the roles that they play in providing strong data handling and the communication of services. Over the last few years, lightweight, cloud-native, and distributed systems have gained more and more traction, and approaches like Spring Boot, Quarkus, and Micronaut have helped to accelerate their development, containerization, and scaling. Furthermore, the use of new tools, such as Resilience4j, Open Telemetry, and Micro Profile, enhances the observability, resilience, and DevOps alignment of the enterprise world. There is a special section with a microservices architecture checklist that focuses on the configuration management, fault tolerance, service discovery, automation of testing, and data consistency. This study gives a detailed insight into how Java is still undergoing development as a flexible, stable and scalable platform to spearhead innovation in web engineering and enable scalable, secure and cloud-native enterprise applications by incorporating a historical and technical analysis.

**Keywords:** *Java web technologies; spring framework; hibernate; enterprise java; microservices; web development.*

## 1. INTRODUCTION

Web technologies have transformed the software engineering environment due to their rapid development (Zhou & Zhang, 2018). Java has also become one of the most versatile and enduring applications. Java, since its beginning, has been a platform that has been stable to enable the development of reliable, scalable and secure web applications. Java is able to adapt and develop to the requirements of the industry and can be used in both simple applications and complex enterprise applications. The paper is a discussion of Java web technologies and best practices that have, over time, developed, starting with servlets and then to microservices.

Java web technologies have a history and best practices have been a part of it, beginning with the servlets of the past century to the microservices of the present ecosystem (Bayani, 2013). So that they can know the characteristics of Struts and Spring structures, the difficulties, and the opportunities where the Java web society has been developed. It also indicates how best practices have evolved over the years to enable Java-based solutions to offer more efficient, reusable, and modular solutions to the increasing challenges of web applications in modernity.

The usability of Java Enterprise Edition (JEE) was expedited as a result of enterprise demands and needs, which necessitated a multitier architecture in enterprise setups (Baldini et al., 2017). Additionally, JEE frameworks allowed the

business world to deal with the complexity of related applications, including best practices, scalability, transaction management and integration with relational models, in which concurrency and data consistency are assured (Sipek et al., 2020). All these innovations essentially affected the design of large systems such that they could be separated in terms of business logic and presentation layers thus reducing complexity and enhancing flexibility. Table 1 below summarizes the evolution of the Java web technologies in a systematic order, which presents a chronological overview of the key frameworks, best practices, and related trade-offs in different development eras.

With the development of cloud computing and distributed computing, microservices have transformed the application of Java in web development (Adilapuram & Engineer, 2025). The lightweight containers, including Docker, and the RESTful web services, made microservices possible through the ability to deploy cross-cutting services flexibly and continuously, assigning them independently and at any point (Arcuri, 2017). Although Java microservices have become a broad and interrelated environment of distributed, polyglot, web-based services (Mudassir & Mushtaq, 2024), the common Java frameworks have developed a clear-cut microservice development strategy. The most popular frameworks of Java microservices are Spring Boot and Spring Cloud (Neeli, 2019). This shift in Java web development is an indication of a shift in the way of thinking about development and deployment of web applications, which

elaborates upon a framework on how to specify autonomous, resilient and independent services.

Java Web Development originality was based on servlets and Java Server Pages (JSP). These technologies facilitated less complicated server-side coding and enhanced dynamic content delivery. The existence of servlets and JSP had placed developers in a state of transitioning off the use of static web applications and onto to the interactive and data-driven applications (Doyle & Lopes, 2005). Java also swiftly spawned other frameworks like Struts, Hibernate and Spring that developed a systematic method of web application development

The significance of the study to the scientific community is that it will provide a systematic and thorough review of how Java web technologies have evolved since the implementation of the Servlets and JSP to the current microservices architectures. This research, by combining some of the important frameworks that include Struts, Spring, Hibernate, and Spring Boot, explains the technological advancements that have defined scalability, modularity, and flexibility in contemporary web systems. The paper facilitates the connection between classical enterprise models and present-day cloud-native ones, providing both historical and practical knowledge, which contributes to the study and education of software engineering. It is also a good resource for practitioners and researchers of modernization, migration, and optimization of legacy Java-based web applications.

## 1.1 Outline of the Paper

The paper is organized as follows: Section II discusses early Java web technologies like Servlets and JSP; Section III explores the evolution of enterprise frameworks such as Spring and Hibernate; Section IV covers modern Java development with microservices and RESTful APIs; Section V highlights advanced enterprise capabilities including CI/CD and containerization; Section VI provides a microservices architecture checklist; Section VII presents a literature review; and Section VIII concludes with key insights and future research directions.

## 2. EARLY JAVA WEB TECHNOLOGIES

One of the basic concepts employed in designing and planning data system building is software development or framework enhancement in programming building (Singh, 2017). Java improvement technique is divided into four. Within the e-learning system structure, an analysis of historical texts of data displays and success leads to the identification of six elements of progress, namely, framework quality, data quality, benefit quality, utilization, client satisfaction, and net benefit, which are summarized in a detailed presentation of the success (Zavala-Romero et al., 2014). The Development Method is presented in Fig. 1 in the central part, and four of them are supported by Practical Knowledge (real-world expertise), Robust Logic (structured reasoning), New Techniques (innovative practices), and Continuous Practices: Broad Room Discussion (continuous practice refinement).

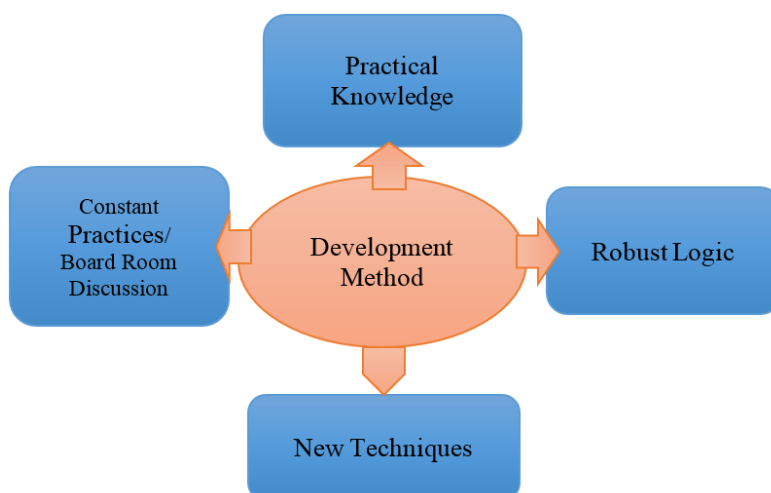


Fig. 1. Java development

## 2.1 JDBC Architecture

The Java Database Connectivity (JDBC) For database access, API supports both two-tier and three-tier processing models; however, the JDBC Architecture typically has two layers (Walimbe, 2018):

**JDBC API:** This provides the application-to-JDBC Manager connection.

**JDBC Driver API:** This supports the JDBC Manager-to-Driver Connection.

The JDBC API offers seamless connection to heterogeneous databases through the use of database-specific drivers and driver management. The JDBC driver manager ensures that each data source is accessed using the appropriate driver (Al-Debagy & Martinek, 2018). The driver management can support multiple drivers connected to various heterogeneous databases concurrently. Fig. 2 of the architecture below illustrates the location of driver

management in relation to the Java application and JDBC drivers.

## 2.2 Java Server Pages (JSP)

Java Server Pages (JSPs) add Java source code to markup documents so that they may be used, for example, to dynamically add application-specific data to a Web page. Template text, which can be in any format, makes up a JSP. This is often HTML code, although it can also be in other forms (WML, TeX, etc.). Additionally, a JSP uses so-called script lets to include Java source code. The embedded Java statements are executed in response to each HTTP request made by a JSP (Dunkel et al., 2004). The markup page is updated with their return values, and the entire page is then sent back to the client. The outcome of combining Java statements with template text is source code that is complicated to understand. To eliminate the Java statements, they might be extracted into certain classes that are accessible in a JSP using so-called custom tags.

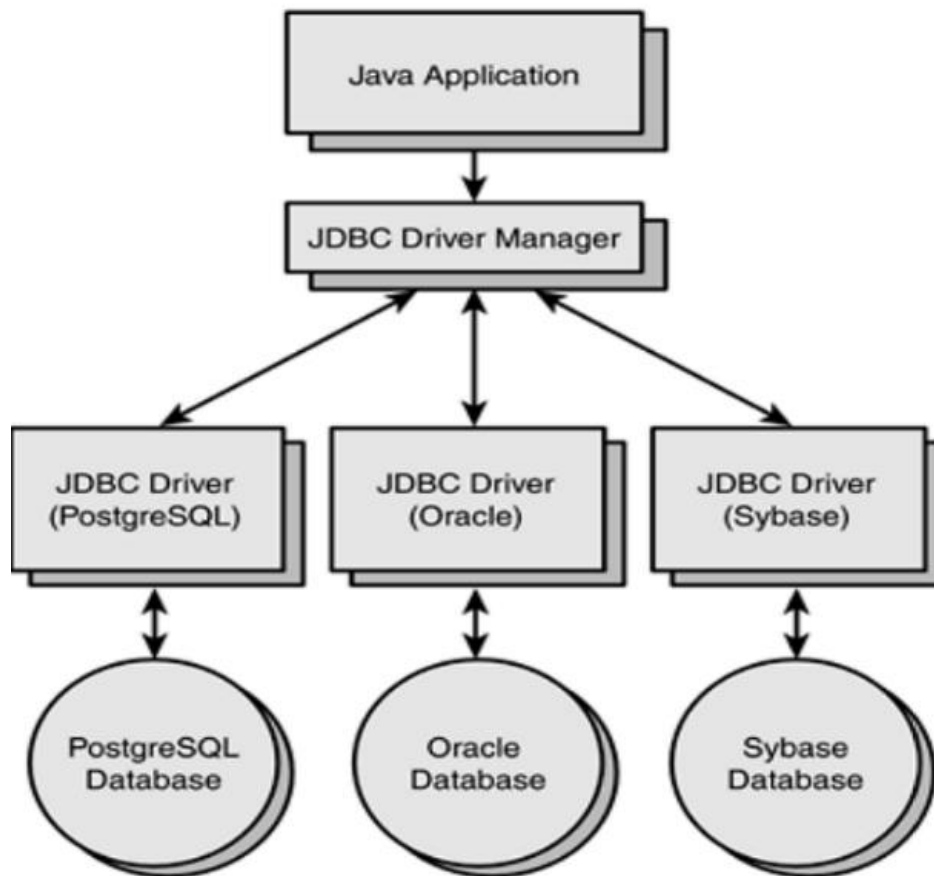
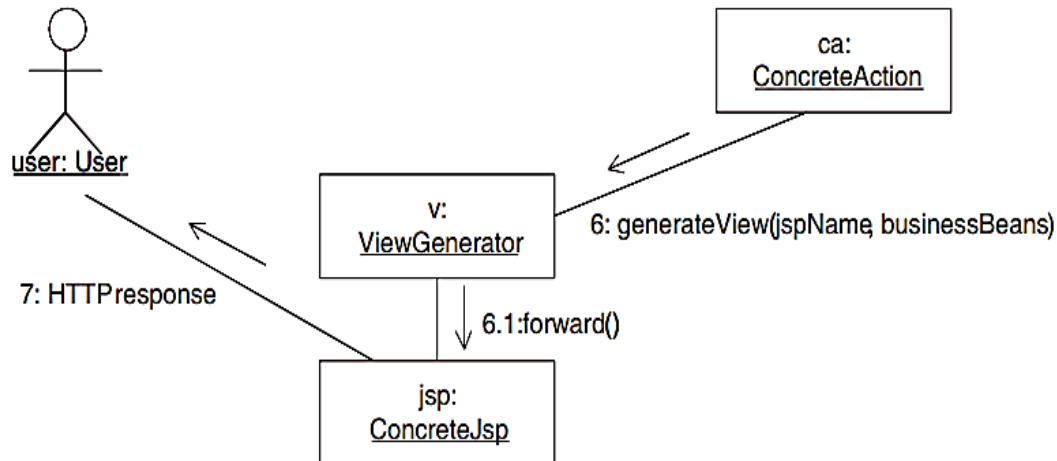


Fig. 2. JDBC architecture



**Fig. 3. Universal architecture with JSPs**

The concrete action class in the JSP variation is in charge of choosing the JSP and exchanging data with the business logic. Because JavaBeans may be used in a simpler format by a JSP, they are typically used for data interchange. The dynamic content can also be sent to a JSP in other methods, for as by including data in the HTTP request object. The control flow of an HTTP request using JSPs is depicted in Fig. 3.

### 2.3 Early Web Frameworks

As web applications grew in complexity, the limitations of Servlets and JSP became evident (Sultan, 2017). Developers required frameworks that could simplify application development, enforce design patterns, and provide reusable components. This need led to the emergence of early Java web frameworks, such as Apache Struts and Java Server Faces (JSF).

**Struts 1.x:** Apache Struts, introduced in 2000, was among the first widely adopted Java web frameworks. It implemented the Model-View-Controller (MVC) design paradigm, which divides

presentation and business logic, as well as navigation. Struts provided the ability to configure centrally via `struts-config.xml`, reusable UI tags, and supported form handling. The applications made it a risk to growth because of its XML-based configuration and the absence of flexibility.

**Java Server Faces (JSF):** An excursion of a Jakarta EE standard, the Java Server Faces (JSF) was introduced as a framework of user interface components. It provided a re-usable User Interface, event processing and support of tools like Face lets. Unlike JSF which reduced boilerplate compared to the Servlets and JSP, its high level of abstraction and high entry barrier reduced its adoption in the agile development environment.

These problems notwithstanding, Struts and JSF defined the foundation of enterprise worthy frameworks and led to the development of more advanced frameworks, including Spring MVC, that, ultimately, became more efficient in terms of scalability and productivity of developers.

**Table 1. Evolution timeline of java web technologies**

| Era          | Frameworks / Technologies                     | Best Practices                                 | Trade-offs                                      |
|--------------|-----------------------------------------------|------------------------------------------------|-------------------------------------------------|
| 1999–2005    | Servlets, JSP, Struts                         | MVC design, separation of concerns             | Manual configuration, limited scalability       |
| 2006–2014    | Spring, Hibernate                             | Dependency Injection, ORM, modularization      | XML configuration overhead                      |
| 2015–2020    | Spring Boot, JPA, REST Easy                   | Microservices, containerization, RESTful APIs  | Increased testing and deployment complexity     |
| 2021–Present | Quarkus, Micronaut, Jakarta EE, Micro Profile | Cloud-native development, CI/CD, observability | Orchestration and service management challenges |

### 3. ENTERPRISE JAVA AND FRAMEWORK EVOLUTION

Enterprise Java has been developed starting with Enterprise JavaBeans (EJB). At the most basic level, EJB provided a standard on both distributed and transactional applications. However, being an individual who rolled it into production and helped customers with its implementation, it did not work in a myriad of ways (Richardson, 2006). This however culminated in the development of the Spring Framework, which was lightweight and flexible in comparison to EJB and other frameworks in JEE, introducing Inversion of Control (IoC) and Dependency Injection (DI), which motivated developers to develop modular applications, and eventually provided a new pattern of application development. Spring provided the first MVC architecture for web applications and allowed for more flexibility and change from earlier frameworks of the time, such as Struts and JSF. With SOA emerging around the same time, the rise of RESTful Web Services, using JAX-RS implementations like Jersey and REST Easy, allowed for scalable, stateless, and interoperable communication with a loosely coupled application model. The shift from EJB to Spring and REST allowed for the creation of a modern microservices-based system.

#### 3.1 Enterprise JavaBeans (EJB)

The business logic of an application is encapsulated in server-side components called enterprise JavaBeans. For instance, an enterprise bean may access a relational database for banking applications or compute interest payments for a loan application. A remote invocation across a network may occur when a client requesting information contacts business methods in the EJB. EJBs provide several advantages. They make it easier for bean developers to create huge applications with dispersed components, to start. Each bean deployed in a J2EE application server receives system-level services from the EJB container, as you may remember. Because of this, bean developers are spared from dealing with intricate problems like resource pooling, transaction management, security, and multithreaded programming. Rather, their bean approaches are only focused on business logic. EJBs help clients at the presentation level as well. Because they employ EJBs for their

business processes and solely perform display logic, clients get slimmer. Additionally, EJBs are portable components, meaning that they may be used to create various corporate applications and run them on any J2EE application server.

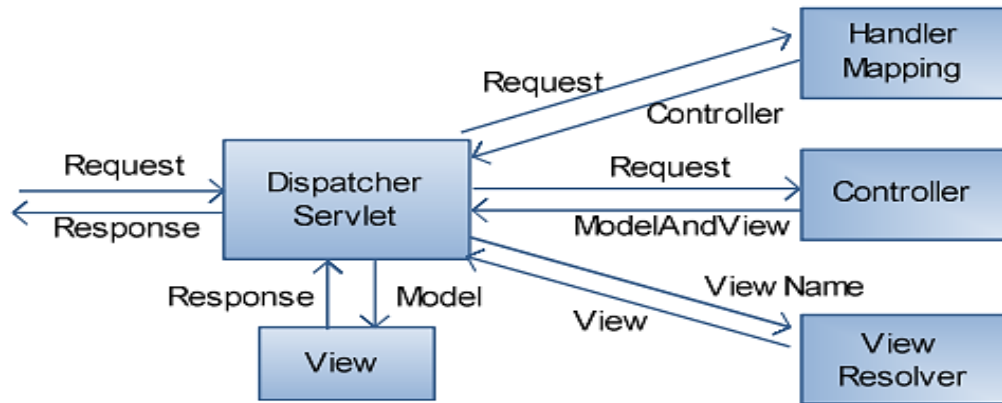
Over time, the EJB model evolved from the heavyweight EJB 2.x specification to the more developer-friendly EJB 3.x, which introduced annotations, dependency injection, and integration with CDI (Contexts and Dependency Injection) and JPA (Java Persistence API). This modernization simplified enterprise development and aligned with the broader shift in Jakarta EE toward a more lightweight, annotation-driven programming model.

#### 3.2 MVC Architecture in Spring

Spring is a lightweight container framework with seven clearly defined modules that combine aspect-oriented programming (AOP) with inversion of control (IoC) features. One of the seven modules, Spring MVC, is a feature-rich, low-weight implementation of MVC for creating Web applications (Zhang et al., 2013). Spring MVC perfectly implements the core concepts and features of MVC with its dynamically configured components by simple configuration. This is the working of Spring MVC whereby when a client sends an HTTP request to the company the dispatcher servlet which forms the main controller, checks the handle mapping according to the URL and directs the request to the appropriate controller. Once the request has been processed by making a call to the business logic in the business logic, the Controller sends the name of the view, the Model, and the View object to the Dispatcher Servlet. View Resolver then converts the model and View provided to the correct view. Fig. 4 is the Spring MVC Request-Handling Pipeline.

#### 3.3 RESTful Web Services

The transformation of the old SOAP-based services to the RESTful Web Services was a great breakthrough in the development of Java web services (Giessler et al., 2015). Representational State Transfer (REST) is an architectural model based on the use of the common HTTP methods of GET, POST, PUT and DELETE to work with resources modeled by URIs.



**Fig. 4. Request-handling pipeline of spring MVC**

Transition from SOAP to REST: SOAP to REST Enterprise applications often used SOAP (Simple Object Access Protocol) due to its standard communications and protocol support. Nonetheless, it was too complicated, XML based, and overbearing to use in lightweight, web based, systems. REST was more popular as being simpler, stateless, and scalable, especially suitable to distributed systems and microservices.

JAX-RS and Jersey: Java API of ROST ful Web Services, or JAX-RS, is the primary component of the Java ecosystem that is used to develop RESTful services and it has annotations that allow easing the definition of resources and handling of requests. Keynote implementations Jersey, and REST Easy eased the task of developers to create REST endpoints with little configuration. These libraries were in favor of the JSON and XML serialization to facilitate smooth data communication between the clients and servers.

RESTful web services therefore became a core component of the current Java web frameworks and micro-services architecture, which make it interoperable, scalable, and easy to integrate with front-end applications and third-party systems.

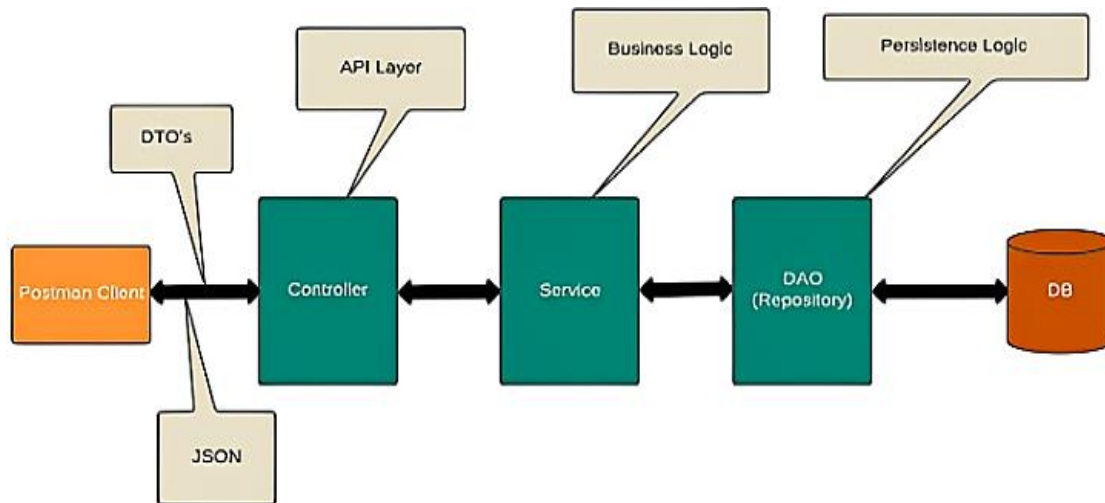
### 3.4 Persistence Frameworks: JPA vs My Batis

Java Persistence API (JPA) offers a standard Object-relational mapping (ORM) model to manage data in relationships within the Java applications. It makes the operations of the databases simple by using entity mapping, which does not require writing the SQL statements explicitly. My Batis, by comparison, provides a semi-ORM programming model which also allows

developers to have full control over the SQL query but maintains object mapping. Although JPA supports fast development and maintainability, My Batis is usually used where query optimization or more complicated interactions with the databases are necessary. These two frameworks have both impacted persistence techniques in contemporary enterprise Java applications.

## 4. MODERN JAVA WEB DEVELOPMENT AND MICROSERVICES

The Java web development has shifted focus towards lightweight frameworks and cloud-ready architecture (Lwakatere et al., 2019). The Spring Boot was also made a favorite by making configurations easy, with an embedded Server, and fast deployment, making it a perfect fit in microservices (Namiot & Sneps-snepe, 2014). Microservices architecture separates the applications into separate and independently deployable services that interact using the RESTful APIs, whereas in the monolithic architecture, all the functionality of the application is incorporated in a single unit. Other frameworks, including Play and Vert.x, proposed reactive programming models, event-driven programming models, which made applications designed to support a large number of simultaneous workloads more efficient and scalable. The implementation of containerization tools, including Docker, and orchestration tools, including Kubernetes has contributed to enhancing portability, resilience, and automation. All of these new concepts challenged the Java programming paradigm and the technology landscape alike to shift towards a more powerful foundation for developing distributed, scalable, and cloud-native applications.



**Fig. 5. Spring boot application architecture**

## 4.1 Spring Boot Application Architecture

The database, Postman, Controller, Service, and DAO (Data Access Object) are all crucial parts of a Spring Boot application that help create the organized and useful backend shown in Fig. 5.

### 4.1.1 Postman

The Spring Boot application does not include Postman, but is an excellent external tool used to test and interact with the RESTful API of the application. Another reason why it is simpler to ensure the functionality of the API is that Postman also lets developers send an HTTP request to the API endpoints of the application. Postman allows testing various HTTP operations (GET, POST, PUT and DELETE) and checking the feedback returned, which will make it a vital tool in the course of API construction and debugging.

### 4.1.2 Controller

The Controller layer is the layer in a Spring boot application that receives the incoming HTTP requests and processing of the application data flow and operations. The labels on the controllers are annotated by Spring, including annotated controller or REST controller annotation, which allows the controllers to receive and send the responses (Kasaram, 2018). Request mappings are defined, and what methods are to be implemented on particular HTTP methods and endpoints (URI). To accept data, process requests, and provide outcomes to the client,

controllers and the service layer communicate with each other.

### 4.1.3 Service

Between the Controller and the DAO (Data Access Object) layers, there is the Service layer. It consists of service actions, application specific rules, and business logic. Services are known as Spring-managed components as they are annotated with annotation `@Service`. They facilitate the separation of concerns by separating the business logic and the web layer. The aggregation and processing are normally undertaken at the service layer before being transmitted to the DAO in which it is used to perform database operations.

## 4.2 Other Modern Frameworks

Besides Spring Boot, there were a few contemporary Java frameworks that came out in the rise of the demand of lightweight, high performance, and developer friendly web applications and services (Xing et al., 2019). The Play Framework and Vert.x are two such examples.

**Play Framework:** Play is a reactive web application framework which is based on the ruby on Rails and was built to support cloud-based, modern applications. It focuses on productivity of developers with functionalities like hot-reloading, Akka asynchronous programming, and support of the REST. Play has a non-blocking, stateless model and is therefore ideal to applications that need scalability and real-time interactions.

**Vert.x:** Vert.x is a platform of reactive apps based on JVM. Due to non-blocking and event-driven nature, it allows developers to handle a large number of requests simultaneously with reduced resources. Vert.x can be used to provide messaging, clustering, and microservices modules and supports multiple programming languages, such as Java, Kotlin, and Groovy. Its polyglot and lightweight features make it very flexible to distributed systems.

These frameworks are used to supplement the traditional Jakarta EE and Spring designs to provide responsiveness, reactivity, and scalability. They became popular in applications that require less efficiency with conventional synchronous models, like streaming platforms, messaging systems and IoT applications.

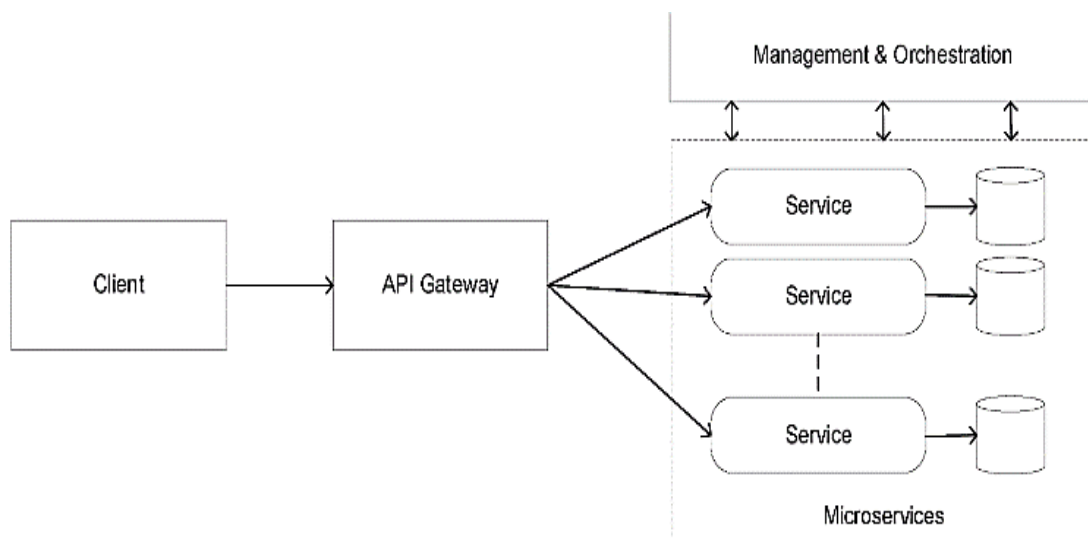
### 4.3 Microservices Architecture

A method for building a single application out of several little services that each run in a different process and communicate with one another via simple means, usually an HTTP API, is the definition of microservices architectures that is most commonly used. Unlike monoliths, microservices can be designed with multiple technology stacks and provide independent deploy ability and scalability (Garriga, 2018). However, this term may also be used for RESTful or conventional services, which contributes to the controversy surrounding SOA and microservices. Despite being a development of SOA, microservices vary fundamentally in how they

handle sharing and reuse. The concept of promoting reuse, or a share-as-much-as-possible architecture style, is the cornerstone of service-oriented architecture (SOA). In contrast, microservices architecture is founded on the notion of a share-as-little-as-possible architecture style. A microservices architecture with an API Gateway is shown in Fig. 6, through which the client interacts, managing routing, authentication, and load balancing. Every microservice is responsible only with a particular function where it has its own database, the orchestration tools such as Kubernetes take care of deployment, monitoring and scaling, making the system more flexible, fault-tolerant and scalable than monolithic architectures.

## 5. ADVANCED ENTERPRISE CAPABILITIES IN MODERN JAVA

The rise of modern enterprise level Java development has outgrown the old monolithic and servlet-based frameworks and is adopting a microservice based paradigm of development that is geared towards scalability, fault tolerance and continuous delivery (Matcha & Kumar, 2025). Modern versions like Eclipse Micro Profile, Resilience4j, Open Telemetry and Micrometer have also emerged as important elements to attain enterprise-level resilience and observability. The technologies allow companies to build cloud-native, container-ready, and Kubernetes-optimized Java applications based on the principles of DevOps and CI/CD.



**Fig. 6. Microservices architecture**

### 5.1 Micro Profile Architecture and Core Capabilities

Eclipse Micro Profile is an open-source standard that adds Jakarta EE to meet the demands of microservice-based architectures (Zhao & Li, 2024). It offers APIs, runtime support of Config, Health, Metrics, Fault Tolerance, Open API, JWT Propagation, and Rest Client among others.

- Micro Profile Config allows dynamic and externalized configuration management without redeploying applications, supporting multiple configuration sources such as system properties, environment variables, and configuration servers.
- Micro Profile Health specifies standard endpoints exposing the liveness and readiness state of services that is critical to orchestrating tools such as Kubernetes to do health checks and rolling updates.
- Micro Profile Metrics enables collection of application-level and system-level performance metrics, supporting integration with monitoring platforms such as Prometheus and Grafana.
- Micro Profile Fault Tolerance provides annotations such as `@Retry`, `@CircuitBreaker`, `@Timeout`, and `@Fallback`, allowing developers to implement robust fault-handling mechanisms declaratively.

With these characteristics, Micro Profile provides a base of cloud-native Java that facilitates observability, flexibility of configuration and automated recovery in distributed systems.

### 5.2 Resilience and Fault Tolerance in Distributed Systems

Resilience forms the core of the stability of any modern system that is based on microservices. A library like Resilience4j is getting more and more popular as it is a more modular and lighter solution than the old ones like Hystrix (Jaiswal, 2025). Resilience4j is a resource for circuit breaking, rate limiting, retry mechanisms, and bulkhead isolation that are the methods that stop the failure of the cascade of the adjacent services.

As an example, the circuit breaker pattern enables the service to recognize the point of failures and so, it stops sending the requests to the unstable components for some time, which in turn makes the whole system more stable. In the

same way, bulkhead isolation makes sure that the malfunction in one service will not eat up all the shared resources and thus, the performance will be preserved for other services. These mechanisms are the base of both HA (high availability) and fault isolation that can be found in large-scale environments deployed in the cloud and written in Java.

### 5.3 Observability and Monitoring

Observability in a distributed environment is the ability to measure, monitor, and debug the systems in such a way that it is effective. These systems being interoperable with containers and microservices make the need for better tool to understand the behavior and performance of the system at runtime more pressing. Open Telemetry and Micrometer are two such frameworks that offer a standardized way of metrics recording, tracing, and logging, thereby giving a complete insight into the runtime performance.

Open Telemetry standardizes distributed tracing and metrics collection, allowing developers to capture request flows across multiple microservices and visualize latency bottlenecks.

Micrometer acts as an instrumentation facade that can smoothly connect with Java frameworks (Spring Boot, Quarkus, Micronaut) and monitoring systems like Prometheus, Influx DB, and Datadog.

Such observability tools form the core for Service-Level Objective (SLO) tracking as well as for root cause analysis, thus they are instrumental in maintaining the reliability and operational transparency that are prerequisites of contemporary enterprise systems.

### 5.4 DevOps and CD Integration

DevOps pipelines that focus on automation, consistency, and iterative deployment are becoming more and more a part of modern Java enterprise development. The frameworks such as Quarkus, Spring Boot, and Micronaut have the features that facilitate containerization through Docker, deployment to Kubernetes, and integration with CI/CD tools like Jenkins, GitLab CI, and GitHub Actions.

Micro Profile and Resilience4j are in line with these DevOps practices by guaranteeing that the services which are deployed will be stable,

**Table 2. Microservices architecture checklist for java-based systems**

| Category                        | Key Focus                                                          | Representative Technologies/Practices             | Description                                                                               |
|---------------------------------|--------------------------------------------------------------------|---------------------------------------------------|-------------------------------------------------------------------------------------------|
| Configuration Management        | Centralized and versioned configuration                            | Spring Cloud Config, Micro Profile Config, Consul | Externalize configuration for portability and maintain consistency across environments.   |
| Service Discovery               | Dynamic service registration and lookup                            | Eureka, Consul, Kubernetes DNS                    | Enables services to locate each other dynamically without hardcoded endpoints.            |
| API Gateway                     | Unified entry point for routing, rate limiting, and authentication | Spring Cloud Gateway, Zuul, Kong                  | Manages request routing, load balancing, and cross-cutting concerns for external clients. |
| Resilience and Fault Tolerance  | Circuit breaking, retries, and fallback strategies                 | Resilience4j, Micro Profile Fault Tolerance       | Ensures high availability by isolating failures and maintaining graceful degradation.     |
| Observability                   | Monitoring, tracing, and logging                                   | Micrometer, Prometheus, Open Telemetry, Jaeger    | Provides visibility into system health and inter-service interactions.                    |
| Testing and Automation          | CI/CD integration, contract and integration testing                | JUnit 5, Test containers, Wire Mock, Jenkins      | Promotes automated quality assurance and continuous delivery pipelines.                   |
| Security                        | Authentication, authorization, and secret management               | Spring Security, OAuth 2.0, Keycloak, Vault       | Protects services through token-based authentication and encrypted secret storage.        |
| Data Management and Consistency | Distributed transactions and eventual consistency                  | Outbox Pattern, Saga Pattern, Event Sourcing      | Ensures reliable data propagation across services while maintaining consistency.          |

**Table 3. Summary of a study on evolution of java web technologies and best practices**

| Author                           | Study On                                           | Approach                                                             | Key Findings                                                                      | Challenges                                                 | Future Directions                                               |
|----------------------------------|----------------------------------------------------|----------------------------------------------------------------------|-----------------------------------------------------------------------------------|------------------------------------------------------------|-----------------------------------------------------------------|
| Akinyede (2018)                  | MVC in Large-Scale Dynamic Web Applications        | MVC-based design in Java Web Architecture                            | MVC ensures modular, reusable, and flexible web applications                      | Partitioning complexity during the design phase            | Simplify MVC integration with microservices and DevOps          |
| Raghavendra & Pushpanjali (2018) | Web Design Frameworks & Hibernate                  | Review of reusable design structures and ORM frameworks              | Hibernate improves persistence; MVC with EJB and Struts enhances flexibility      | Complexity in N-tier frameworks                            | Optimize ORM performance; integrate with modern frameworks      |
| Majeed & Rauf (2018)             | MVC Layers in Web Development                      | Analytical study of MVC functionalities and practices                | Detailed explanation of MVC layers, advantages, and best practices                | Limited focus on integration with new frameworks           | Extend MVC practices with microservices and reactive frameworks |
| Lupaşcu (2017)                   | Java Web Technologies for Project Management Tools | Implemented client-server system using Servlets, JSP, and JavaScript | Java Web tech improves usability and decision-making in project management        | Integration with modern UI technologies                    | Enhance with responsive design and cloud support                |
| Pahl & Jamshidi (2016)           | Microservices in Cloud Computing                   | Systematic mapping study of 21 papers                                | Classified and compared microservices research; identified cloud suitability      | Lack of consolidated secondary studies; evolving practices | Refine taxonomies, extend studies with containerization trends  |
| D. D. Sarkar & A. Jaiswal (2015) | MVC in Java Web Applications                       | Multi-framework integration using J2EE for enterprise applications   | Separation of business and presentation logic improves scalability and efficiency | Complexity in handling multiple frameworks                 | Explore lightweight frameworks and improved integration methods |

configurable, and observable even when the iteration cycles are very rapid (Pulugulla, 2025). Besides that, automated monitoring through Open Telemetry creates very fast feedback loops that have an impact on deployment quality and operational efficiency.

### 5.5 Modular Monolith vs. Microservices: Strategic Considerations

Even though microservices architectures allow scaling, parallel development, and fault isolation, they still complicate deployment, monitoring, and inter-service communication (Dhakade et al., 2024). In the case of smaller systems or those with business logic that is tightly coupled, a modular monolith can be a more manageable and cheaper solution.

With a modular monolith, a project keeps separation of concerns by having clearly defined modules and layered architecture, however, it functions within a single deployable unit. Therefore, the move to microservices must be determined by the intricacy of the system, the need for scalability, and the maturity of the team in handling distributed systems. Companies are progressively choosing a “monolith-first” approach, gradually moving to microservices only when domain-driven decomposition and operational scaling require it.

### 6. MICROSERVICES ARCHITECTURE CHECKLIST IN THE JAVA ECOSYSTEM

The adoption of microservices in the Java ecosystem introduces a set of architectural, operational, and design considerations that ensure scalability, maintainability, and resilience. A structured checklist helps developers and architects verify that each service is independently deployable, observable, secure, and consistent within a distributed environment. Table 2 summarizes the essential dimensions of a robust Java-based microservices architecture, covering configuration management, service discovery, API gateway design, resilience mechanisms, observability, testing strategies, security practices, and data consistency models.

Modern Java runtimes such as Graal VM and OpenJ9 further optimize microservice

performance by supporting native image compilation, which reduces startup latency and memory footprint. For deployment, container-native build tools such as Jib and Cloud Native Build packs simplify packaging Java microservices into OCI-compliant images, while layered JARs enable efficient rebuilds in continuous integration pipelines. Adopting these practices ensures alignment with cloud-native standards and operational excellence in distributed environments.

## 7. LITERATURE REVIEW

The reviewed literature highlights the progressive evolution of Java web technologies, with an emphasis on MVC frameworks, design patterns, enterprise solutions, and microservices. Collectively, these studies address scalability, modularity, and best practices, while identifying challenges and proposing future research directions.

Thakur and Pandey (2019) the most widely used and effective programming paradigm for creating dynamic, large-scale online applications. MVC enables developers to create software that is modular, reusable, and adaptable by utilizing design patterns that are often recognized as solutions for recurring issues. The MVC design pattern is consequently more difficult to use in Web applications since modern technologies urge developers to divide the application as early as the design stage. An online program developed using the MVC framework and Java Web Architecture (Thakur & Pandey, 2019).

Raghavendra and Pushpanjali (2018) the usage of web design frameworks as a conceptual approach to improve the likelihood of Web application reuse. It first illustrated the necessity of developing reusable and abstract directive design frameworks using a range of Web information systems. For countless databases, the Hibernate Framework Technology is a unique and organized resource. It also describes how to provide object-oriented systems with permanent features. It provides a denotation of both and shows how design patterns and frameworks relate to one another. Java supports the MVC Model's N-tier framework in addition to EJB, Struts WEB Framework, and Hibernate technologies (Raghavendra & Pushpanjali, 2018).

Majeed and Rauf (2018) the most important information about MVC layers, applications, benefits, and best practices for developing web apps. Describe the functions of each of the three levels in detail. The main objective of the study is to provide a thorough comprehension of every MVC layer, as well as the primary features and benefits of the MVC. In particular, describe the recommended practices for developing web apps based on MVC. Additionally, describe the associated technologies that are utilized with web applications that are built on MVC (Majeed & Rauf, 2018).

Lupaşcu (2017) the benefits of developing internet technologies for project management software and the effects they have on people and organizations. Consequently, an information system for project management is based on and implements a client-server architecture. JavaScript is utilized on the client side of the system to build dynamic web pages and enhance the user experience. In contrast, the server side uses Java Web Technologies, such as JSP and servlets. The purpose of this project management information system is to assist managers in making decisions throughout the project planning process, coordination, and supervision (Lupaşcu, 2017).

Pahl and Jamshidi (2016) The microservices architectural method seems to be a wonderful fit for container technology at the PaaS level. However, there isn't yet a secondary study to gather this data. This research aims to determine, classify, and compare the body of research on microservices and their use in the cloud. From the beginning of the microservices pattern two years ago till the end of 2015, they conducted thorough mapping study on twenty-one selected studies. A characterization framework was used to classify and compare the selected publications (Pahl & Jamshidi, 2016).

Sarkar et al. (2015) This framework is used to construct MVC, which separates presentation logic from business logic. There are three different Java platforms, or flavors, that each cater to different programming requirements. J2EE provides a multitier architecture environment for developing business services or applications. In an increasingly demanding technological world, scalable, efficient, and fast information management solutions are becoming more and more important. It is possible to meet

these demands with JEE technology, which is suitable. This study has demonstrated that building a Java application using multiple frameworks and MVC concepts is easier than using a single framework. More and more study papers have been published on the latest trends that bypass frameworks and develop Web architecture using a quick and innovative method (Sarkar et al., 2015).

Table 3 is a chronological synthesis of review studies on Java web technologies describing approaches, results, difficulties, and future directions, in which a shift in evolution is evident towards microservices starting with servlets.

## 8. CONCLUSION AND FUTURE WORK

The evolution of Java web technologies reflects the broader transformation of software engineering toward modularity, scalability, and cloud-native development. From the early days of Servlets, JSP, and JDBC to modern frameworks such as Spring Boot, Quarkus, and Micronaut, Java has consistently adapted to meet the changing demands of enterprise applications. The adoption of enterprise frameworks such as Struts, Spring, Hibernate, and EJB made the systems more maintainable, reusable, and efficient, whereas the recent changes in containerization, microservices, and DevOps integration have made Java a leading platform for distributed systems. Moreover, the inclusion of such tools as Resilience4j, Open Telemetry, and Micro Profile has helped to improve the observability, performance, and resilience of sophisticated architectures. As the future step, research and development may decide to work on the integration of artificial intelligence (AI) and machine learning (ML) into Java-based systems to facilitate predictive analytics, adaptive resource management, and intelligent automation. Furthermore, the incorporation of Global Sensitivity Analysis (GSA) can enhance performance evaluation in dynamic environments. Expanding Java's interoperability with edge computing, IoT, and serverless architectures can also create new possibilities for high-performance, real-time systems. Continued exploration of security, sustainability, and lightweight frameworks will further ensure that Java remains a robust, efficient, and future-ready platform for the evolving landscape of enterprise and web application development.

## DISCLAIMER

Author(s) hereby declare that generative AI technologies such as Large Language Models, etc have been used during writing or editing of this manuscript. This explanation will include the name, version, model, and source of the generative AI technology and as well as all input prompts provided to the generative AI technology.

## COMPETING INTERESTS

Authors have declared that they have no known competing financial interests or non-financial interests, or personal relationships that could have appeared to influence the work reported in this paper.

## REFERENCES

- Adilapuram, S., & Engineer, S. (2025). Optimizing Microservices and Serverless Applications with Micronaut: A High-Performance Alternative to Traditional Java Frameworks. *International Journal of Engineering Science Research and Technology*, 14(6), 1–6.
- Al-Debagy, O., & Martinek, P. (2018, November). A Comparative Review of Microservices and Monolithic Architectures. In *2018 IEEE 18th International Symposium on Computational Intelligence and Informatics (CINTI)* (pp. 000149–000154). <https://doi.org/10.1109/CINTI.2018.8928192>
- Akinyede, R. (2018). An E-Commerce Model-View-Controller (MVC) Framework For A Cloud Based Environment. <https://doi.org/10.9790/0050-05022028>
- Arcuri, A. (2017). RESTful API automated test case generation. *Proceedings of the 2017 IEEE International Conference on Software Quality, Reliability and Security (QRS 2017)*, (1), 9–20. <https://doi.org/10.1109/QRS.2017.11>
- Baldini, I., et al. (2017). Serverless computing: Current trends and open problems. In *Research Advances in Cloud Computing*. [https://doi.org/10.1007/978-981-10-5026-8\\_1](https://doi.org/10.1007/978-981-10-5026-8_1)
- Bayani, M. (2013). Web-based Decision Support Systems: A conceptual performance evaluation. *Proceedings of the 2013 IEEE 17th International Conference on Intelligent Engineering Systems (INES)*, 6(2), 21–26. <https://doi.org/10.1109/INES.2013.6632824>
- Dhakade, P. A., Tajane, D., Borkar, P., Battawar, A., & Wankhade, V. (2024). Java: Evolution, Features, and Its Impact on Modern Software Development. *International Journal of Novel Research and Development*, 9(8), 2456–4184.
- Doyle, B., & Lopes, C. V. (2005). Survey of Technologies for Web Application Development. 2(3), 1–43.
- Dunkel, J., Bruns, R., & Holitschke, A. (2004). Comparison of JavaServer Pages and XSLT: A software engineering perspective. *Software: Practice and Experience*, 34(1), 1–13. <https://doi.org/10.1002/spe.555>
- Garriga, M. (2018). Towards a taxonomy of microservices architectures. *Lecture Notes in Computer Science*, 203–218. [https://doi.org/10.1007/978-3-319-74781-1\\_15](https://doi.org/10.1007/978-3-319-74781-1_15)
- Giessler, P., Gebhart, M., Sarancin, D., Steinegger, R., & Abeck, S. (2015). Best Practices for the Design of RESTful Web Services. *International Conference on Software Advances*, 392–297.
- Jaiswal, I. A. (2025, March). Integrating AI into enterprise Java applications for secure high performance and scalable systems. *International Journal of Computational and Experimental Science and Engineering*, 11(4), 7653–7662. <https://doi.org/10.22399/ijcesen.4086>
- Kasaram, C. R. (2018). Spring Boot and Microservices: Engineering Modular and Scalable Back-End Architectures. *International Journal of Computer Science Engineering Research and Development (IJCSE RD)*, 9(1), 1–10.
- Lupaşcu, A. (2017, May). Project Management Information System Based on Web Technologies. *Edu World 7th International Conference*, 1008–1015. <https://doi.org/10.15405/epsbs.2017.05.02.124>

- Lwakatare, L. E., et al. (2019). DevOps in practice: A multiple case study of five companies. *Information and Software Technology*, 114, 217–230.  
<https://doi.org/10.1016/j.infsof.2019.06.010>
- Majeed, A., & Rauf, I. (2018). MVC Architecture: A Detailed Insight to the Modern Web Applications Development. *Peer Review Journal of Solar Photoenergy Systems*, 1(1), 1–7.
- Matcha, S., & Kumar, S. (2025). Java/J2EE Development: Best Practices and Performance Optimization in Enterprise Applications. *International Journal of Scientific Development and Research*, 10(1), b123–b138.
- Mudassir, M., & Mushtaq, M. (2024). The role of APIs in modern software development. *World Journal of Advanced Engineering Technology and Sciences*, 13(1), 1045–1047.  
<https://doi.org/10.30574/wjaets.2024.13.1.0515>
- Namiot, D., & Sneps-snepe, M. (2014). On Micro-services Architecture. *International Journal of Open Information Technologies (ISSN)*, 2(9), 24–27.
- Neeli, S. S. S. (2019). Serverless Databases: A Cost-Effective and Scalable Solution. *International Journal of Innovative Research in Engineering and Multidisciplinary Physical Sciences*, 7(6), 1–7.
- Pahl, C., & Jamshidi, P. (2016). Microservices: A systematic mapping study. *Proceedings of the 6th International Conference on Cloud Computing and Services Science (CLOSER 2016)*.  
<https://doi.org/10.5220/0005785501370146>
- Pulugulla, M. R. (2025). Engineering Cloud-Native Microservices in Java: A Scalable Approach for Modern Enterprise Software. *ICONIC Research and Engineering Journals*, 9(3), 1–10.
- Raghavendra, G., & Pushpanjali, P. (2018). A Review on Java Frameworks for Web Applications. *International Journal of Scientific Research in Computer Science, Engineering and Information Technology (JSRCSEIT)*, 3(3), 615–619.
- Richardson, C. (2006, June). Untangling Enterprise Java. *Queue*, 4(5), 36–44.  
<https://doi.org/10.1145/1142031.1142045>
- Sarkar, D. D., Jaiswal, A., & Saxena, A. (2015). Understanding architecture and framework of J2EE using web application. *International Journal of Computer Science and Information Technology*, 6(2), 1253–1257.
- Singh, T. (2017). A Survey on Java Programming Language and Methods of Improvisation. *International Journal of Innovative and Advanced Computer Science*, 6(12).
- Sipek, M., Muharemagic, D., Mihaljevic, B., & Radovan, A. (2020). Enhancing performance of cloud-based software applications with GraalVM and Quarkus. *Proceedings of the 2020 43rd International Convention on Information, Communication and Electronic Technology (MIPRO 2020)*, 1746–1751.  
<https://doi.org/10.23919/MIPRO48935.2020.9245290>
- Sultan, M. (2017). Angular and the trending frameworks of mobile and web-based platform technologies: A comparative analysis. *Future Technologies Conference*, 928–936.
- Thakur, R. N., & Pandey, U. S. (2019). A Study Focused on Web Application Development using MVC Design Pattern. *International Research Journal of Engineering and Technology*, 6(8), 237–244.
- Walimbe, M. P. (2018). Review on Java Database Connectivity. *International Research Journal of Engineering and Technology*, 5(3), 549–550.
- Xing, Y. K., Huang, J. P., & Lai, Y. Y. (2019). Research and analysis of the front-end frameworks and libraries in e-business development. *ACM International Conference Proceeding Series*, 68–72.  
<https://doi.org/10.1145/3313991.3314021>
- Zavala-Romero, O., Ahmed, A., Chassignet, E. P., Zavala-Hidalgo, J., Fernández Eguiarte, A., & Meyer-Baese, A. (2014, December). An open source Java web application to build self-contained web GIS sites. *Environmental Modelling & Software*, 62, 210–220.  
<https://doi.org/10.1016/j.envsoft.2014.08.029>
- Zhang, D., Wei, Z., & Yang, Y. (2013). Research on lightweight MVC framework based on Spring MVC and MyBatis. *Proceedings of the 6th International Symposium on Computational Intelligence and Design (ISCID 2013)*, 1, 350–353.  
<https://doi.org/10.1109/ISCID.2013.94>
- Zhao, X., & Li, X. (2024). The Latest Trends and Challenges in Enterprise Application Development with Java. *Advances in Computer and*

Signal Systems, 8(2), 44–50. <https://doi.org/10.23977/acss.2024.080207>  
Zhou, Y., & Zhang, S. (2018). Research on the Web Application System Development Course Group for the Training of Innovation Talents. *2nd International Conference on Education Science and Economic Management*, 184, 515–519. <https://doi.org/10.2991/icesem-18.2018.119>

**Disclaimer/Publisher's Note:** The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of the publisher and/or the editor(s). This publisher and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.

© Copyright (2025): Author(s). The licensee is the journal publisher. This is an Open Access article distributed under the terms of the Creative Commons Attribution License (<http://creativecommons.org/licenses/by/4.0>), which permits unrestricted use, distribution, and reproduction in any medium, provided the original work is properly cited.

Peer-review history:

The peer review history for this paper can be accessed here:  
<https://pr.sdiarticle5.com/review-history/146611>